The Family Menu

Database Design

Andrew Connally
03/30/2026

CHANGE/UPDATE NOTE: I've highlighted the stretch feature, in Section 3-B. No other changes were made.

Database Technology: MongoDB

Primary Goal: Store meals, meal plans, and generated grocery lists

Core Business Logic: Ingredient aggregation, grocery list generation, and estimated cost calculation

---

1. Database Technology Choice

MongoDB is an appropriate choice for *The Family Menu* because the application primarily works with self-contained records that map naturally to JSON documents.

- A meal already contains the bounded data needed to display a recipe card: a title, description, ingredients, instructions, image, and estimated cost information.
- MongoDB lets the application store and retrieve this data in the same structure that the Node.js and Express backend will already use when sending JSON responses to the frontend.

The application's most common workflows are document-oriented rather than relational.

- Users browse meals, select meals to add to an overall meal plan, and then a grocery list is generated from that meal plan.

- This will not require a large number of many-to-many joins. Instead, it benefits from fast retrieval of whole meal documents and simple references from one document to another.

MongoDB also fits the business logic described in the project proposal.

- The backend services perform ingredient aggregation, grocery list generation, and cost estimation.

- These operations are easier when recipe data is stored in nested arrays, because each meal document can carry its own ingredient list and instruction steps.

- This keeps reads straightforward while still allowing references where reuse makes more sense, such as linking a meal plan to reusable meal documents.

Why MongoDB is a better fit than a relational database for this project

Project Characteristic	Why MongoDB fits
Meal data is naturally grouped	**A meal can embed its ingredients and instructions in one document.**
Ingredient lists are finite and bounded	**Embedding works well because one meal only has a manageable number of ingredients.**
Frontend and API exchange JSON	**MongoDB stores BSON/JSON-style documents that closely match API payloads.**
Meal plans reuse the same meals	**Referencing meal ID's avoids copying full meal data into every meal plan.**
Grocery lists are generated outputs	**A grocery list document can persist the calculated result for later viewing without expensive recomputation on every page load.**

2. Embedding vs. Referencing Strategy

Embedding
The best practice in MongoDB is to embed data when it is always used together, has a limited size, and belongs loosely to one parent document.

- *Ingredients* and *instructions* should be embedded inside each meal document because they are part of the meal itself and are always needed when a recipe card or recipe details page is shown.

- A meal is not meaningful without its ingredients and instructions.

Referencing
The best practice in MongoDB is to reference data when it is reused across many documents, updated independently, or would cause duplication problems if embedded everywhere.

- *Meal plans* should reference meals by mealId rather than embedding full meal details. This avoids duplicating recipe data in every individual meal plan, allows meal details to be managed in one place, and keeps meal plan documents small.

- *Grocery lists* should reference the meal plan they were generated from, but the aggregated grocery items themselves should be embedded because that list is the calculated output for one specific plan.

Relationship strategy summary

Structure	Strategy	Reason	Update Approach
meals -> ingredients	Embed	Ingredients are part of the meal and fetched together	Update inside the meal document
meals -> instructions	Embed	Instruction steps belong only to that meal	Update inside the meal document
mealPlans -> meals	Reference by key identifier	Same meal can appear in many plans	Store mealId and query meal documents when needed

groceryLists -> items	Embed	Items are generated specifically for one grocery list	Regenerate or replace embedded items when meal plan changes
groceryLists -> mealPlans	Reference by key identifier	Items are generated specifically for one grocery list	Update reference only if a new plan/list is created

3. Collection Design

The database will contain four primary collections: users, meals, mealPlans, and groceryLists. Each structure below lists the fields, data types, and any key identifiers.

A. users collection

```
{
  "_id": ObjectId,
  "name": "string",
  "email": "string",
  "passwordHash": "string",
  "createdAt": "date",
  "updatedAt": "date"
}
```

Purpose and role in the application

The users collection stores the person using the application. While the wireframes focus on a single persona, the application still needs a way to separate one user's meal plan from another user's meal plan. The user document provides that ownership boundary.

A user can create or maintain one or more meal plans over time. The mealPlans collection will reference the user by userId.

B. meals collection

```
{
  "_id": ObjectId,
  "title": "string",
  "description": "string",
  "imageUrl": "string",
  "estimatedMealCost": "number", // Derived from adding up
of estimatedUnitCost in ingredients
  "ingredients": [
    {
      "ingredientId": "string", // Will need to ensure that
the same base ingredient isn't given multiple
ingredientId's across multiple recipes
      "name": "string",
      "quantity": "number",
      "unit": "string",
      "estimatedUnitCost": "number"
    }
  ],
  "instructions": ["string"],
  "tags": ["string"], // EX: ["quick", "kid-friendly",
"low-carb"] - can be fully utilized if search function is
created
  "createdAt": "date",
  "updatedAt": "date"
}
```

Purpose and role in the application

The meals collection is the core content collection for the application. It powers the Meal Library page, the recipe card view, and the recipe instruction overlay shown in the wireframes.

Embedding ingredients and instruction steps inside each individual meal document is the correct choice because those values are always displayed together. This matches the project's need to browse meals and immediately show recipe details without joining multiple collections. All the information for a meal can be retrieved from a single database query.

The ingredientId field is not a key to another collection in this design. Instead, it acts as a normalization aid inside aggregation logic so the service layer can recognize when two ingredients represent the same grocery item. For example, two meals that both contain 'broccoli' can be merged into one grocery list line item.

C. mealPlans collection

```
{
  "_id": ObjectId,
  “userId”: “ObjectId”,  // key identifier -> users._id
  "status": "string", // draft, finalized, archived. Lets
users save plans & remove old ones
  "meals": [
    {
      "mealId": "ObjectId", // key identifier -> meals._id
      "servingsOverride": "number|null" // possible
addition: lets users select number of servings they want
the meal to create, and auto-adjusts recipe
ingredients/recipe instructions
    }
  ],
  "estimatedTotalCost": "number",
  "createdAt": "date",
  "updatedAt": "date"
}
```

Purpose and role in the application

The mealPlans collection stores a user's selected meals for a specific meal plan. This directly supports the My Meal Plan page shown in the wireframes and storyboard.

The meals array stores references rather than full meal objects. That is the better design here because the same meal may appear in many different users' plans or in the same user's plans across multiple meal plans. Referencing avoids repeated copies and keeps recipe maintenance centralized.

The estimatedTotalCost field can be stored for quick display on the planner page, while the grocery list service can still recalculate totals whenever meals are added or removed. estimatedTotalCost is derived from the sum of all estimatedMealCost from the individual selected meals that were added to the mealPlan collection.

D. groceryLists collection

```
{
  "_id": ObjectId,
  "mealPlanId": "ObjectId", // key identifier ->
mealPlans._id
  "userId": "ObjectId", // key identifier -> users._id
  "items": [
    {
      "ingredientId": "string",
      "name": "string",
      "totalQuantity": "number",
      "unit": "string",
      "estimatedLineCost": "number",
      "sourceMealIds": ["ObjectId"] // key identifiers ->
meals._id
    }
  ],
  "estimatedTotalCost": "number",
  "generatedAt": "date",
  "updatedAt": "date" }
```

Purpose and role in the application

The groceryLists collection stores the consolidated shopping list generated from a meal plan. This supports the My Grocery List page and preserves the exact result of the application's aggregation logic.

The items array is embedded because a grocery list is a single generated output tied to one meal plan. Those line items are not reused independently elsewhere.

Storing sourceMealIds inside each grocery line item gives the application traceability. This makes it possible to explain where an ingredient came from, debug aggregation logic, or regenerate the list when the plan changes.

---

4. How the Data Structures Are Used in the Application

Application Feature	Collections Used	How the Data Is Used
Meal Library	**meals**	**Reads full meal documents to display recipe cards with image, prep time, ingredients summary, and cost information.**
Recipe Instruction Overlay	**meals**	**Reads the instructions array from the selected meal document and renders each step in order.**
Add meal to meal plan	**mealPlans + meals**	**Stores a mealId reference inside the meals array of the selected meal plan.**
Remove meal from meal plan	**mealPlans**	**Deletes the selected meal reference from the meals array and triggers grocery list recalculation.**
Generate grocery list	**mealPlans + meals + groceryLists**	**Loads meal ids from the plan, reads each meal's ingredients, merges duplicates, calculates quantities and costs, then saves the generated list document.**

Display estimated cost	**mealPlans + groceryLists**	**Returns the saved estimatedTotalCost to the planner and grocery list pages for quick feedback to the user.**

5. Example Usage Scenario

A user, such as Busy-Mom Emma:

1) Opens the Meal Library and browses the meals collection.
2) She adds two meals to her meal plan.

The application then:

3) Writes those meal ID's into the current mealPlans document for that meal plan.

The service layer will then:

4) Read both referenced meal documents, loop through their embedded ingredients arrays, group matching ingredients together, sum the quantities, and calculate estimated line-item and total cost.
5) The result is saved as a groceryLists document tied to the selected meal plan.

When Busy-Mom Emma opens My Grocery List, the application does not need to rebuild the list from scratch unless the plan has changed. It can simply read the stored groceryLists document and render the consolidated items and estimated total cost.

---

6. Update and Consistency Considerations

Because mealPlans reference meal documents, editing a meal updates the source recipe in one place (the meals collection). Future grocery lists generated from that recipe will automatically use the newest meal data. However, current, already-generated grocery lists and meal plans will not automatically update.

- This behavior is intentional, as users would have chosen and saved meals for their plan based on what recipe, ingredients, and estimated costs they were currently viewing.

However, if a meal is edited by the developers *after* it was already added to a meal plan by a user, but *before* the grocery list is generated, the finalized grocery list would be based on the updated meal document, because the meal plan references the current meal document rather

than storing an older copy of the recipe.

- The meal document contains the ingredients and instructions, which is where the grocery list and recipe card would generate its information from.

- While the user would see the correct, updated recipe card to match the generated grocery list, it needs to be considered that they made selections based on what information about the meal they were currently viewing, not what it was later updated to.

If preserving historical accuracy becomes important, the application could snapshot selected meal titles or ingredient summaries inside groceryLists at generation time.

When a meal is removed from a plan, the backend should either regenerate the linked grocery list document or delete and recreate it. This keeps the stored grocery list consistent with the current meal plan.

---

7. Conclusion

MongoDB is the appropriate database technology for *The Family Menu* because the project is centered on JSON-style documents, bounded on-to-many embedded data, and service-layer logic that transforms meal documents into meal plans and grocery lists. The design uses embedding where data naturally belongs together and references where reuse and maintainability matter. As a result, the database structure directly supports the application's meal browsing, planning, grocery aggregation, and cost estimation features.